November 2, 2026 at 19:05

1. Introduction. A decomposition of a rectangle into t subrectangles is *incomparable* if none of the pieces would fit inside another—not even after a quarter turn. Exercise 7.2.2.1–374 develops the theory; exercise 375 asks for the extremes.

> **375.** [*M29*] Among all the incomparable dissections of order (a) seven and (b) eight, restricted to integer sizes, find the rectangles with the smallest possible semiperimeter (height plus width). Also find the smallest possible squares that have incomparable dissections in integers. *Hint:* Show that there are 2^t potential ways to mix the h's with the w's, preserving their order; and find the smallest semiperimeter for each of those cases.

Answer 375 solves this with linear programming: fix the way the heights and the widths interleave, and what remains is a small LP in the eight (or nine) independent dimensions. This program does the same thing, exactly, over the rationals—but it starts a step further back, by enumerating the possible dissections instead of taking them from the answer.

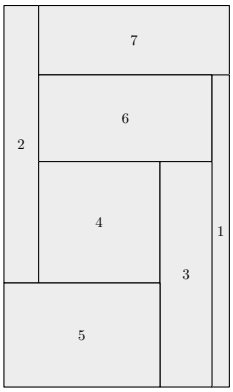
2. Every number in answer 375(a) came out, and so did every number in (b) but one. The four diagrams of (b) are said to have best semiperimeters $(44, 44, 44, 56)$; the fourth is 47. A 30×17 rectangle can be cut into the eight incomparable pieces

$$4 \times 16, \quad 5 \times 13, \quad 9 \times 9, \quad 10 \times 8, \quad 11 \times 7, \quad 12 \times 6, \quad 17 \times 3, \quad 20 \times 1,$$

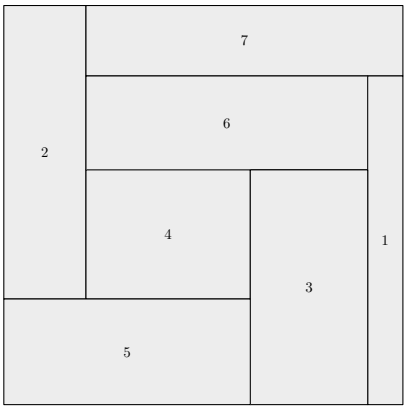
which is that diagram, and $30 + 17 = 47$. It changes no other answer: three of the four diagrams still reach 44, so 44 is still the smallest semiperimeter with eight subrectangles.

I wrote this in September 2026 while reading the exercise and its answer carefully. These notes, with the numbers the program produced, are the companion document `README.md` in the directory above.

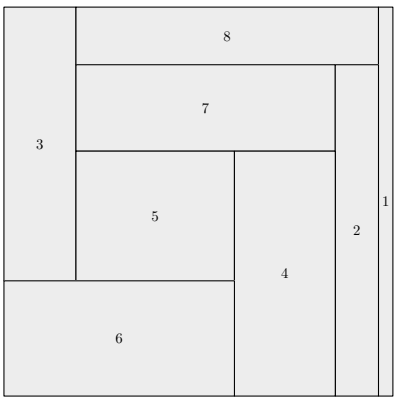
3. Four dissections, drawn to their own scales. The first is the unique rectangle of semiperimeter 35 with seven incomparable pieces, and the second is the unique square, of side 34. The third is the smallest square with eight pieces, of side 27. The fourth is the 30×17 rectangle above.



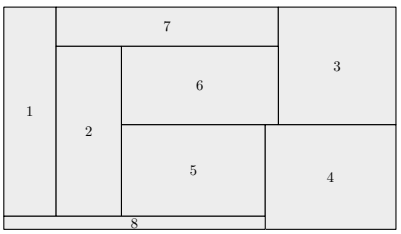
13×22, seven pieces



34×34, seven pieces



27×27, eight pieces



30×17, eight pieces

4. The skeleton is a command line and a choice of what to check.

```
package main
import (
    "flag"
    "fmt"
    "log"
    "math/big"
    "sort"
    "strings"
    cells "github.com/sjnam/dancing-cells"
)
⟨ Declarations 6 ⟩
⟨ Functions 7 ⟩
func main() {
    ⟨ Read the command line 5 ⟩
    for _, m := range modes {
        switch m {
        case "motley":
            ⟨ Count the motley dissections, exercise 365 23 ⟩
        case "census":
            ⟨ Take a census of the patterns of orders 7 and 8 27 ⟩
        case "seven":
            ⟨ Work out order seven 68 ⟩
        case "eight":
            ⟨ Work out order eight 76 ⟩
        case "nine":
            ⟨ Ask whether nine subrectangles could do better 86 ⟩
        case "direct":
            ⟨ Search the patterns directly, with no theory at all 96 ⟩
        default:
            log.Fatalf("unknown mode %q", m)
        }
    }
}
```

5. ⟨ Read the command line 5 ⟩ ≡

```
mode := flag.String("mode", "all",
    "motley, census, seven, eight, nine, direct, or all")
flag.Parse()
modes := []string{*mode}
if *mode ≡ "all" {
    modes = []string{"motley", "census", "seven", "eight", "nine", "direct"}
}
```

This code is used in section 4.

6. Dissections and their reductions. The *reduction* of a decomposition, defined in exercise 360, distorts it until it fits an $m \times n$ grid with every one of the coordinates $\{0, 1, \ldots, m\}$ and $\{0, 1, \ldots, n\}$ used by at least one boundary. A pattern equal to its own reduction is *reduced*. It is *strictly* reduced (exercise 362) if no subrectangle cuts all the way across, and *motley* (exercise 365) if in addition no two subrectangles are cut off by the same pair of horizontal or of vertical lines.

Incomparability forces most of that. Two pieces with the same pair of vertical lines have equal widths, hence one fits inside the other; so the reduction of an incomparable dissection always has distinct intervals. It need not be strict, though—answer 374(b) makes the point—so this program keeps the non-strict patterns too, and finds that they matter.

⟨ Declarations 6 ⟩ ≡

```
type rect struct { r0, r1, c0, c1 int }   // rows [r0..r1), columns [c0..c1)
type pattern struct {
    m, n int
    rs []rect
}
```

This code is also defined in sections 32, 47, 48, 64.

This code is used in section 4.

7. Every dissection of the grid, found by covering the first empty cell in every possible way. The *limit* stops the recursion once a tiling has more pieces than we can use.

⟨ Functions 7 ⟩ ≡

```
func tilings(m, n, limit int) []pattern {
    var out []pattern
    var cur []rect
    cell := make([][]int, m)
    for i := range cell {
        cell[i] = make([]int, n)
        for j := range cell[i] {
            cell[i][j] = -1
        }
    }
    var rec func()
    rec = func() {
        if len(cur) > limit {
            return
        }
        ⟨ Find the first empty cell, or record a tiling 8 ⟩
        ⟨ Try every rectangle with that cell at its corner 9 ⟩
    }
    rec()
    return out
}
```

This code is also defined in sections 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 25, 29, 31, 33, 35, 37, 39, 41, 42, 43, 44, 45, 46, 49, 50, 52, 54, 60, 62, 63, 65, 88, 90, 91, 92, 93, 94, 95.

This code is used in section 4.

8. ⟨ Find the first empty cell, or record a tiling 8 ⟩ ≡

```
r0, c0 := −1, −1
for i := 0; i < m ∧ r0 < 0; i++ {
    for j := 0; j < n; j++ {
        if cell[i][j] < 0 {
            r0, c0 = i, j
            break
        }
    }
}
if r0 < 0 {
    out = append(out, pattern{m, n, append([]rect(Λ), cur...)})
    return
}
```

This code is used in section 7.

9. ⟨ Try every rectangle with that cell at its corner 9 ⟩ ≡

```
for r1 := r0 + 1; r1 ≤ m; r1++ {
    if r1 > r0 + 1 ∧ cell[r1 − 1][c0] ≥ 0 {
        break
    }
    for c1 := c0 + 1; c1 ≤ n; c1++ {
        if ¬free(cell, r0, r1, c0, c1) {
            break
        }
        paint(cell, r0, r1, c0, c1, len(cur))
        cur = append(cur, rect{r0, r1, c0, c1})
        rec()
        cur = cur[:len(cur) − 1]
        paint(cell, r0, r1, c0, c1, −1)
    }
}
```

This code is used in section 7.

10. ⟨ Functions 7 ⟩ +≡

```
func free(cell [][]int, r0, r1, c0, c1 int) bool {
    for i := r0; i < r1; i++ {
        for j := c0; j < c1; j++ {
            if cell[i][j] ≥ 0 {
                return false
            }
        }
    }
    return true
}
```

11. ⟨ Functions 7 ⟩ +≡

```
func paint(cell [][]int, r0, r1, c0, c1, v int) {
    for i := r0; i < r1; i++ {
        for j := c0; j < c1; j++ {
            cell[i][j] = v
        }
    }
}
```

12. The three filters.

⟨ Functions 7 ⟩ +≡

```
func (p pattern) reduced() bool {
    usedR, usedC := make([]bool, p.m + 1), make([]bool, p.n + 1)
    for _, r := range p.rs {
        usedR[r.r0], usedR[r.r1] = true, true
        usedC[r.c0], usedC[r.c1] = true, true
    }
    for _, b := range append(usedR, usedC...) {
        if ¬b {
            return false
        }
    }
    return true
}
```

13. ⟨ Functions 7 ⟩ +≡

```
func (p pattern) distinctIntervals() bool {
    sr, sc := map[[2]int]bool{}, map[[2]int]bool{}
    for _, r := range p.rs {
        if sr[[2]int{r.r0, r.r1}] ∨ sc[[2]int{r.c0, r.c1}] {
            return false
        }
        sr[[2]int{r.r0, r.r1}], sc[[2]int{r.c0, r.c1}] = true, true
    }
    return true
}
```

14. ⟨ Functions 7 ⟩ +≡

```
func (p pattern) strict() bool {
    for _, r := range p.rs {
        if (r.r0 ≡ 0 ∧ r.r1 ≡ p.m) ∨ (r.c0 ≡ 0 ∧ r.c1 ≡ p.n) {
            return false
        }
    }
    return true
}
```

15. Patterns come in families of up to eight, under the rotations and reflections of the rectangle, and the eight members all pose the same question. So each is reduced to a canonical form and counted once.

⟨ Functions 7 ⟩ +≡

```
func (p pattern) canon() string {
    best, cur := "", p
    for k := 0; k < 4; k++ {
        for _, q := range []pattern { cur, cur.flip() }{
            if s := q.normal(); best == "" ∨ s < best {
                best = s
            }
        }
        cur = cur.rot()
    }
    return best
}
```

16. ⟨ Functions 7 ⟩ +≡

```
func (p pattern) rot() pattern {
    q := pattern{p.n, p.m, Λ}
    for _, r := range p.rs {
        q.rs = append(q.rs, rect{r.c0, r.c1, p.m − r.r1, p.m − r.r0})
    }
    return q
}
```

17. ⟨ Functions 7 ⟩ +≡

```
func (p pattern) flip() pattern {
    q := pattern{p.m, p.n, Λ}
    for _, r := range p.rs {
        q.rs = append(q.rs, rect{r.r0, r.r1, p.n − r.c1, p.n − r.c0})
    }
    return q
}
```

18. ⟨ Functions 7 ⟩ +≡

```
func (p pattern) normal() string {
    rs := append([]rect(Λ), p.rs...)
    sort.Slice(rs, func(i, j int) bool {
        if rs[i].r0 ≠ rs[j].r0 {
            return rs[i].r0 < rs[j].r0
        }
        return rs[i].c0 < rs[j].c0
    })
    s := fmt.Sprintf("%dx%d:", p.m, p.n)
    for _, r := range rs {
        s += fmt.Sprintf("(%d,%d,%d,%d)", r.r0, r.r1, r.c0, r.c1)
    }
    return s
}
```

19. A pattern prints as its grid of letters, one per piece, which is also how the diagrams of the answers are transcribed below.

⟨ Functions 7 ⟩ +≡

```
func (p pattern) String() string {
    g := make([][]byte, p.m)
    for i := range g {
        g[i] = make([]byte, p.n)
    }
    for k, r := range p.rs {
        paintByte(g, r, byte('a' + k))
    }
    var b strings.Builder
    for i, row := range g {
        if i > 0 {
            b.WriteByte('/')
        }
        b.Write(row)
    }
    return b.String()
}
```

20. ⟨ Functions 7 ⟩ +≡

```
func paintByte(g [][]byte, r rect, c byte) {
    for i := r.r0; i < r.r1; i++ {
        for j := r.c0; j < r.c1; j++ {
            g[i][j] = c
        }
    }
}
```

21. A dissection of order t whose reduction is $m \times n$ has $t = m + n - 1$ unless some four pieces meet at a point, and a point where four meet gives two pieces the same corner—so for us $m + n \le t + 1$ always. That bound keeps the census small.

⟨ Functions 7 ⟩ +≡

```
func patterns(t int, needStrict bool) []pattern {
    var out []pattern
    seen := map[string]bool{}
    for m := 1; m ≤ t; m++ {
        for n := 1; m + n ≤ t + 1; n++ {
            for _, p := range tilings(m, n, t) {
                ⟨ Keep p if it is a pattern we want 22 ⟩
            }
        }
    }
    return out
}
```

22. ⟨ Keep p if it is a pattern we want 22 ⟩ ≡

```
if len(p.rs) ≠ t ∨ ¬p.reduced( ) ∨ ¬p.distinctIntervals( ) {
    continue
}
if needStrict ∧ ¬p.strict( ) {
    continue
}
if c ≔ p.canon( ); ¬seen[c] {
    seen[c] = true
    out = append(out, p)
}
```

This code is used in section 21.

23. The census. Exercise 365 counts the motley dissections: two of size 3×3, sixteen of size 4×4 "8 + 8 solutions", and twenty of size 4×5 "4+4+4+4+2+2". Those are the drawings, before dividing by the symmetries; the essentially distinct ones number 1, 2 and 6. Our enumerator should say the same.

⟨ Count the motley dissections, exercise 365 23 ⟩ ≡

```
fmt.Println("motley␣dissections,␣as␣exercise␣365␣counts␣them")
for _, mn := range [][2]int {{3, 3}, {4, 4}, {4, 5} }{
    m, n := mn[0], mn[1]
    all, red, mot := 0, 0, 0
    seen := map[string]bool{ }
    for _, p := range tilings(m, n, m * n) {
        all++
        if ¬p.reduced( ) {
            continue
        }
        red++
        if ¬p.strict( ) ∨ ¬p.distinctIntervals( ) {
            continue
        }
        mot++
        seen[p.canon( )] = true
    }
    fmt.Printf("␣␣␣␣%d␣x␣%d:␣%7d␣tilings,␣%6d␣reduced,␣%2d␣motley," +
        "␣%d␣up␣to␣symmetry\n", m, n, all, red, mot, len(seen))
    ⟨ Ask ssxcc to enumerate the same tilings 24 ⟩
}
```

This code is used in section 4.

24. The tilings themselves are an exact cover problem—one item per cell, one option per subrectangle—so the engine of this repository can count them as a second opinion. (Exercise 365 asks for something sharper: a construction that makes Algorithm M produce the motley dissections and nothing else.)

⟨ Ask ssxcc to enumerate the same tilings 24 ⟩ ≡

```
x := cells.NewXCC( )
var got int
for range x.Dance(strings.NewReader(gridProblem(m, n))).Solutions {
    got++
}
fmt.Printf("␣␣␣␣␣␣␣␣ssxcc␣agrees␣on␣the␣number␣of␣tilings:␣%v\n", got ≡ all)
```

This code is used in section 23.

25. ⟨ Functions 7 ⟩ +≡

```
func gridProblem(m, n int) string {
    var b strings.Builder
    for i := 0; i < m; i++ {
        for j := 0; j < n; j++ {
            fmt.Fprintf(&b, "c%d.%d␣", i, j)
        }
    }
    b.WriteByte('\n')
    for r0 := 0; r0 < m; r0++ {
        for r1 := r0 + 1; r1 ≤ m; r1++ {
            ⟨ Write the options with rows r0 to r1 26 ⟩
        }
    }
    return b.String()
}
```

26. ⟨ Write the options with rows *r0* to *r1* 26 ⟩ ≡

```
for c0 := 0; c0 < n; c0++ {
    for c1 := c0 + 1; c1 ≤ n; c1++ {
        for i := r0; i < r1; i++ {
            for j := c0; j < c1; j++ {
                fmt.Fprintf(&b, "c%d.%d␣", i, j)
            }
        }
        b.WriteByte('\n')
    }
}
```

This code is used in section 25.

27. For orders 7 and 8 the census is short: only 4×4 and only 4×5 survive, four patterns and eighteen of them, of which two and six are motley. Most of them cannot be made incomparable at any size, and the linear programs say so once and for all—no bound on the dimensions is involved.

⟨ Take a census of the patterns of orders 7 and 8 27 ⟩ ≡

```
for _, t := range []int { 7, 8 }{
    ps, strict := patterns(t, false), patterns(t, true)
    sizes := map[string]int{}
    for _, p := range ps {
        sizes[fmt.Sprintf("%dx%d", p.m, p.n)]++
    }
    live := 0
    fmt.Printf("order␣%d:␣%d␣patterns␣with␣distinct␣intervals␣%v," +
        "␣of␣which␣%d␣are␣motley\n", t, len(ps), sizes, len(strict))
    for _, p := range ps {
        ⟨ Say whether p can be made incomparable 28 ⟩
    }
    fmt.Printf("␣␣␣␣%d␣of␣the␣%d␣support␣an␣incomparable␣dissection\n",
        live, len(ps))
}
```

This code is used in section 4.

28. A pattern is dead unless some labeling and some interleaving of that labeling have a solution.

⟨ Say whether p can be made incomparable 28 ⟩ ≡

```
n, cases := 0, 0
for _, perm := range p.labelings() {
    if ok, _ := p.looseLP(perm, false); ¬ok {
        continue
    }
    for sgn := 0; sgn < 1 ≪ uint(t); sgn++ {
        if ok, _ := p.chain(perm, sgn, false).lpOpt(); ok {
            cases++
        }
    }
    if cases > 0 {
        n++
    }
}
if n > 0 {
    live++
    fmt.Printf("␣␣␣␣%s␣motley=%-5v␣%d␣labelings,␣one␣of␣them␣feasible" +
        "␣in␣%d␣cases\n", p, p.strict(), len(p.labelings()), cases)
}
```

This code is used in section 27.

29. Labelings. Answer 374(a) is the lemma everything rests on. Sort the two dimensions of each piece as $(\min_i, \max_i)$. Incomparability says that $\min_i < \min_j$ forces $\max_i > \max_j$, so sorting the pieces by their minima puts their maxima in the opposite order; every minimum is at most every maximum; and if the $2t$ dimensions in increasing order are $z_1 \le \cdots \le z_{2t}$, then $z_1 < \cdots < z_t \le z_{t+1} < \cdots < z_{2t}$ with the piece of the jth smallest minimum having dimensions $\{z_j, z_{2t+1-j}\}$.

Label the pieces $1, \ldots, t$ so that $w_1 < \cdots < w_t$; then $h_1 > \cdots > h_t$. Which labelings are possible is partly settled by the pattern alone: if one piece uses a subset of another's columns it is no wider, and if it uses a subset of another's rows it is no taller.

⟨ Functions 7 ⟩ +≡

```
func (p pattern) labelings() [][]int {
    t := len(p.rs)
    var out [][]int
    perm, used := make([]int, t), make([]bool, t)
    var rec func(k int)
    rec = func(k int) {
        if k == t {
            out = append(out, append([]int(nil), perm...))
            return
        }
        for i := 0; i < t; i++ {
            ⟨ Give label k + 1 to piece i, if that is allowed 30 ⟩
        }
    }
    rec(0)
    return out
}
```

30. The piece taking the next label must be no wider and no taller than every piece still unlabeled—no wider because the widths increase with the label, no taller because the heights fall.

⟨ Give label $k+1$ to piece i, if that is allowed 30 ⟩ ≡

```
if used[i] {
    continue
}
ok := true
for j := 0; j < t ∧ ok; j++ {
    if used[j] ∨ j ≡ i {
        continue
    }
    if inside(p.rs[j].c0, p.rs[j].c1, p.rs[i].c0, p.rs[i].c1) {
        ok = false   // piece j can never be the wider one
    }
    if inside(p.rs[i].r0, p.rs[i].r1, p.rs[j].r0, p.rs[j].r1) {
        ok = false   // piece i can never be the taller one
    }
}
if ¬ok {
    continue
}
used[i], perm[k] = true, i
rec(k + 1)
used[i] = false
```

This code is used in section 29.

31. ⟨ Functions 7 ⟩ +≡

```
func inside(a0, a1, b0, b1 int) bool {
    return a0 ≥ b0 ∧ a1 ≤ b1 ∧ (a0 ≠ b0 ∨ a1 ≠ b1)
}
```

32. Cases. Now the hint. Each of $z_1, \ldots, z_t$ is either a height or a width, and z_{2t+1-j} is then the other kind; so 2^t sign vectors describe every way the two increasing sequences can interleave. Together a labeling and a sign vector fix the order of all $2t$ dimensions, and each dimension is a sum of the n column widths or of the m row heights. What is left is a chain of linear forms, strictly increasing except in the middle, where $z_t \le z_{t+1}$.

⟨ Declarations 6 ⟩ +≡

```
type problem struct {
    atoms int    // the n column widths, then the m row heights
    cols int    // how many of them are widths
    form []uint32    // the 2t dimensions, as sets of atoms, in order
    gap []int    // form[k] exceeds form[k − 1] by at least this
    square bool    // insist that the two sides be equal
}
```

33. ⟨ Functions 7 ⟩ +≡

```
func (p pattern) chain(perm []int, sgn int, square bool) problem {
    t := len(p.rs)
    role := make([]bool, 2 * t)    // is the kth smallest dimension a width?
    for k := 0; k < t; k++ {
        w := sgn & (1 ≪ uint(k)) ≠ 0
        role[k], role[2 * t − 1 − k] = w, ¬w
    }
    pr := problem{atoms: p.n + p.m, cols: p.n, square: square}
    wi, hi := 0, t − 1
    for k := 0; k < 2 * t; k++ {
        ⟨ Append the kth dimension to the chain 34 ⟩
    }
    return pr
}
```

34. The widths are taken in increasing order of label and the heights in decreasing order, which is what the labeling means.

⟨ Append the kth dimension to the chain 34 ⟩ ≡

```
var mask uint32
if role[k] {
    r := p.rs[perm[wi]]
    for c := r.c0; c < r.c1; c++ {
        mask |= 1 ≪ uint(c)
    }
    wi++
} else {
    r := p.rs[perm[hi]]
    for c := r.r0; c < r.r1; c++ {
        mask |= 1 ≪ uint(p.n + c)
    }
    hi--
}
g := 1
if k ≡ t {
    g = 0
}
pr.form, pr.gap = append(pr.form, mask), append(pr.gap, g)
```

This code is used in section 33.

35. Writing a case out the way the answer writes it makes the two easy to compare. The kth and the $(2t + 1 − k)$th dimensions are of opposite kinds, which is what the second line unfolds.

⟨ Functions 7 ⟩ +≡

```
func caseName(t, sgn int) string {
    var b strings.Builder
    wi, hi := 1, t
    for k := 0; k < 2 * t; k++ {
        bit := k
        if k ≥ t {
            bit = 2 * t − 1 − k
        }
        ⟨ Write the separator before the kth dimension 36 ⟩
        if (sgn & (1 ≪ uint(bit)) ≠ 0) ≡ (k < t) {
            fmt.Fprintf(&b, "w%d", wi)
            wi++
        } else {
            fmt.Fprintf(&b, "h%d", hi)
            hi--
        }
    }
    return b.String()
}
```

36. ⟨ Write the separator before the kth dimension 36 ⟩ ≡

```
if k ≡ t {
    b.WriteString("␣<=␣")
} else if k > 0 {
    b.WriteString("␣<␣")
}
```

This code is used in section 35.

37. The chain becomes a linear program in the obvious way: one row per link, and one more pair of rows if the two sides must agree.

⟨ Functions 7 ⟩ +≡

```
func (pr problem) rows()([]lrow, []int) {
    var rs []lrow
    for k, f := range pr.form {
        a := make([]int, pr.atoms)
        for i := 0; i < pr.atoms; i++ {
            if f & (1 ≪ uint(i)) ≠ 0 {
                a[i]++
            }
            if k > 0 ∧ pr.form[k − 1] & (1 ≪ uint(i)) ≠ 0 {
                a[i]−−
            }
        }
        rs = append(rs, lrow{a, ge, pr.gap[k]})
    }
    ⟨ Add the row that makes it a square 38 ⟩
    c := make([]int, pr.atoms)
    for i := range c {
        c[i] = 1
    }
    return rs, c
}
```

38. The objective is the sum of all the atoms, which is the width plus the height: the semiperimeter. For a square that sum is twice the side.

⟨ Add the row that makes it a square 38 ⟩ ≡

```
if pr.square {
    rs = append(rs, squareRow(pr))
}
```

This code is used in section 37.

39. Before spending 2^t linear programs on a labeling it is worth asking one cheap question: can the widths increase and the heights fall at all, never mind how the two sequences interleave? That is the same program with the interleaving thrown away, and it is a relaxation of every one of the 2^t cases, so a labeling that fails it can be dropped whole. Most are.

⟨ Functions 7 ⟩ +≡

```
func (p pattern) looseLP(perm []int, square bool)(bool, *big.Rat) {
    t := len(p.rs)
    pr := problem{atoms: p.n + p.m, cols: p.n, square: square}
    var rs []lrow
    for k := 0; k < t; k++ {
        w, h := p.mask(perm[k], false), p.mask(perm[k], true)
        ⟨ Say that the widths rise and the heights fall 40 ⟩
    }
    if square {
        pr.form = Λ
        rs = append(rs, squareRow(pr))
    }
    c := make([]int, pr.atoms)
    for i := range c {
        c[i] = 1
    }
    ok, v, _ := solveLP(rs, c)
    return ok, v
}
```

40. ⟨ Say that the widths rise and the heights fall 40 ⟩ ≡

```
rs = append(rs, atLeast(pr.atoms, w, 0, 1))
rs = append(rs, atLeast(pr.atoms, h, 0, 1))
if k > 0 {
    rs = append(rs, atLeast(pr.atoms, w, p.mask(perm[k − 1], false), 1))
    rs = append(rs, atLeast(pr.atoms, p.mask(perm[k − 1], true), h, 1))
}
```

This code is used in section 39.

41. ⟨ Functions 7 ⟩ +≡

```
func (p pattern) mask(piece int, rows bool) uint32 {
    var m uint32
    r := p.rs[piece]
    if rows {
        for c := r.r0; c < r.r1; c++ {
            m |= 1 ≪ uint(p.n + c)
        }
    } else {
        for c := r.c0; c < r.c1; c++ {
            m |= 1 ≪ uint(c)
        }
    }
    return m
}
```

42. The row saying that one sum exceeds another by at least *gap*.

⟨ Functions 7 ⟩ +≡

```
func atLeast(atoms int, plus, minus uint32, gap int) lrow {
    a := make([]int, atoms)
    for i := 0; i < atoms; i++ {
        if plus & (1 << uint(i)) != 0 {
            a[i]++
        }
        if minus & (1 << uint(i)) != 0 {
            a[i]--
        }
    }
    return lrow{a, ge, gap}
}
```

43. ⟨ Functions 7 ⟩ +≡

```
func squareRow(pr problem) lrow {
    a := make([]int, pr.atoms)
    for i := range a {
        a[i] = 1
        if i ≥ pr.cols {
            a[i] = −1
        }
    }
    return lrow{a, eq, 0}
}
```

44. ⟨ Functions 7 ⟩ +≡

```
func (pr problem) lpOpt()(bool, *big.Rat) {
    rs, c := pr.rows()
    ok, v, _ := solveLP(rs, c)
    return ok, v
}
```

45. ⟨ Functions 7 ⟩ +≡

```
func (pr problem) ipOpt(cap int)(bool, int, []int) {
    rs, c := pr.rows()
    return solveIP(rs, c, cap)
}
```

46. Given a solution the widths and heights come straight back, in the numbering the answers use.

⟨ Functions 7 ⟩ +≡

```
func (p pattern) wh(perm, v []int)([]int, []int) {
    w, h := make([]int, len(p.rs)), make([]int, len(p.rs))
    for i, pc := range perm {
        r := p.rs[pc]
        for c := r.c0; c < r.c1; c++ {
            w[i] += v[c]
        }
        for c := r.r0; c < r.r1; c++ {
            h[i] += v[p.n + c]
        }
    }
    return w, h
}
```

47. Exact linear programming. The programs here are tiny—at most eighteen rows and nine real variables—but they decide questions of feasibility, and a feasibility claim made in floating point is not a claim at all. So this is a two-phase simplex over exact rationals, with Bland's rule so that it cannot cycle.

⟨ Declarations 6 ⟩ +≡

```
const (
    ge = iota
    le
    eq
)
type lrow struct {
    a []int
    sense int
    b int
}
```

48. ⟨ Declarations 6 ⟩ +≡

```
type tab struct {
    m, n, N int
    t [][]*big.Rat   // m + 1 by N + 1; the last row holds the reduced costs
    basis []int
    art []bool   // which columns are artificial
}
```

49. ⟨ Functions 7 ⟩ +≡

```
func rat(i int64) *big.Rat { return new(big.Rat).SetInt64(i) }
```

50. ⟨ Functions 7 ⟩ +≡

```
func (T *tab) pivot(r, col int) {
    p := new(big.Rat).Set(T.t[r][col])
    for j := 0; j ≤ T.N; j++ {
        if T.t[r][j].Sign() ≠ 0 {
            T.t[r][j].Quo(T.t[r][j], p)
        }
    }
    for i := 0; i ≤ T.m; i++ {
        if i ≡ r ∨ T.t[i][col].Sign() ≡ 0 {
            continue
        }
        ⟨ Clear column col in row i 51 ⟩
    }
    T.basis[r] = col
}
```

51. ⟨ Clear column *col* in row *i* 51 ⟩ ≡

```
f := new(big.Rat).Set(T.t[i][col])
for j := 0; j ≤ T.N; j++ {
    if T.t[r][j].Sign() ≠ 0 {
        T.t[i][j].Sub(T.t[i][j], new(big.Rat).Mul(f, T.t[r][j]))
    }
}
```

This code is used in section 50.

52. Bland's rule: always take the first column that would help and, among the rows that tie in the ratio test, the one whose basic variable has the smallest index. Slower than the textbook choice, but it terminates.

⟨ Functions 7 ⟩ +≡

```
func (T *tab) run(allowed func(int) bool) {
    for {
        col := −1
        for j := 0; j < T.N; j++ {
            if allowed(j) ∧ T.t[T.m][j].Sign() < 0 {
                col = j
                break
            }
        }
        if col < 0 {
            return
        }
        ⟨ Pivot on the best row of column col, or give up 53 ⟩
    }
}
```

53. ⟨ Pivot on the best row of column *col*, or give up 53 ⟩ ≡

```
row, best := −1, new(big.Rat)
for i := 0; i < T.m; i++ {
    if T.t[i][col].Sign() ≤ 0 {
        continue
    }
    q := new(big.Rat).Quo(T.t[i][T.N], T.t[i][col])
    if row < 0 ∨ q.Cmp(best) < 0 ∨
        (q.Cmp(best) ≡ 0 ∧ T.basis[i] < T.basis[row]) {
        row, best = i, q
    }
}
if row < 0 {
    return   // unbounded, which our objectives never are
}
T.pivot(row, col)
```

This code is used in section 52.

54. Minimise $c \cdot x$ over $x \ge 0$ subject to the given rows.

```
⟨ Functions 7 ⟩ +≡
  func solveLP(rows []lrow, c []int)(bool, *big.Rat, []*big.Rat) {
      m, n := len(rows), len(c)
      T := &tab{m: m, n: n, N: n + 2 * m, basis: make([]int, m)}
      T.art = make([]bool, T.N)
      T.t = make([][]*big.Rat, m + 1)
      for i := range T.t {
          T.t[i] = make([]*big.Rat, T.N + 1)
          for j := range T.t[i] {
              T.t[i][j] = new(big.Rat)
          }
      }
      for i, r := range rows {
          ⟨ Enter row i of the tableau 55 ⟩
      }
      ⟨ Phase I: drive the artificial variables to zero 56 ⟩
      ⟨ Phase II: minimise the objective 58 ⟩
      ⟨ Read off the solution 59 ⟩
  }
```

55. A row whose right-hand side is negative is multiplied through by -1 first, so that every right-hand side is nonnegative and the starting basis is legal. An inequality that then reads $\le$ needs only a slack variable; the others need a surplus and an artificial.

```
⟨ Enter row i of the tableau 55 ⟩ ≡
  sign, sense := 1, r.sense
  if r.b < 0 {
      sign, sense = -1, le + ge - r.sense
      if r.sense ≡ eq {
          sense = eq
      }
  }
  for j := 0; j < n; j++ {
      T.t[i][j] = rat(int64(sign * r.a[j]))
  }
  T.t[i][T.N] = rat(int64(sign * r.b))
  switch sense {
  case ge:
      T.t[i][n + i], T.t[i][n + m + i] = rat(-1), rat(1)
      T.basis[i], T.art[n + m + i] = n + m + i, true
  case le:
      T.t[i][n + i] = rat(1)
      T.basis[i] = n + i
  case eq:
      T.t[i][n + m + i] = rat(1)
      T.basis[i], T.art[n + m + i] = n + m + i, true
  }
```

This code is used in section 54.

56. ⟨ Phase I: drive the artificial variables to zero 56 ⟩ ≡

```
for j := 0; j < T.N; j++ {
    if T.art[j] {
        continue
    }
    acc := new(big.Rat)
    for i := 0; i < m; i++ {
        if T.art[T.basis[i]] {
            acc.Add(acc, T.t[i][j])
        }
    }
    T.t[m][j] = acc.Neg(acc)
}
acc := new(big.Rat)
for i := 0; i < m; i++ {
    if T.art[T.basis[i]] {
        acc.Add(acc, T.t[i][T.N])
    }
}
T.t[m][T.N] = acc.Neg(acc)
T.run(func(j int) bool { return ¬T.art[j] })
if T.t[m][T.N].Sign() ≠ 0 {
    return false, Λ, Λ   // the rows contradict each other
}
⟨ Drive any artificial variable out of the basis 57 ⟩
```

This code is used in section 54.

57. One can be left in the basis at value zero, and then it must be pivoted away before the second phase, or a later pivot might make it positive again.

⟨ Drive any artificial variable out of the basis 57 ⟩ ≡

```
for i := 0; i < m; i++ {
    if ¬T.art[T.basis[i]] {
        continue
    }
    for j := 0; j < T.N; j++ {
        if ¬T.art[j] ∧ T.t[i][j].Sign() ≠ 0 {
            T.pivot(i, j)
            break
        }
    }
}
```

This code is used in section 56.

58. ⟨ Phase II: minimise the objective 58 ⟩ ≡

```
for j := 0; j ≤ T.N; j++ {
    T.t[m][j] = new(big.Rat)
}
for j := 0; j < n; j++ {
    T.t[m][j] = rat(int64(c[j]))
}
for i := 0; i < m; i++ {
    k := T.basis[i]
    if k ≥ n ∨ c[k] ≡ 0 {
        continue
    }
    f := rat(int64(c[k]))
    for j := 0; j ≤ T.N; j++ {
        if T.t[i][j].Sign() ≠ 0 {
            T.t[m][j].Sub(T.t[m][j], new(big.Rat).Mul(f, T.t[i][j]))
        }
    }
}
T.run(func(j int) bool { return ¬T.art[j] })
```

This code is used in section 54.

59. ⟨ Read off the solution 59 ⟩ ≡

```
x := make([]*big.Rat, n)
for j := range x {
    x[j] = new(big.Rat)
}
for i := 0; i < m; i++ {
    if T.basis[i] < n {
        x[T.basis[i]].Set(T.t[i][T.N])
    }
}
return true, new(big.Rat).Neg(T.t[m][T.N]), x
```

This code is used in section 54.

60. The exercise wants integers, and the answer notes that the linear programs "usually have integer solutions; but sometimes they don't." So the integer optimum needs branch and bound: solve the relaxation, and if some variable comes out fractional, split the problem on it and take the better half.

⟨ Functions 7 ⟩ +≡

```
func solveIP(rows []lrow, c []int, cap int)(bool, int, []int) {
    n := len(c)
    best, bestX := cap + 1, []int(Λ)
    var rec func(rs []lrow)
    rec = func(rs []lrow) {
        ok, v, x := solveLP(rs, c)
        if ¬ok {
            return
        }
        if lo := ceilRat(v); lo ≥ best {
            return
        }
        ⟨ Take the solution, or branch on a fractional variable 61 ⟩
    }
    rec(rows)
    if bestX ≡ Λ {
        return false, 0, Λ
    }
    return true, best, bestX
}
```

61. Every coefficient of the objective is 1, so an integral vertex is an integral solution and its value is the bound we already computed.

⟨ Take the solution, or branch on a fractional variable 61 ⟩ ≡

```
frac := -1
for j := 0; j < n ∧ frac < 0; j++ {
    if ¬x[j].IsInt() {
        frac = j
    }
}
if frac < 0 {
    best, bestX = ceilRat(v), make([]int, n)
    for j := range bestX {
        bestX[j] = ceilRat(x[j])
    }
    return
}
for _, br := range [][2]int {{le, floorRat(x[frac])}, {ge, floorRat(x[frac]) + 1} }{
    row := lrow{a: make([]int, n), sense: br[0], b: br[1]}
    row.a[frac] = 1
    rec(append(append([]lrow(Λ), rs...), row))
}
```

This code is used in section 60.

62. ⟨ Functions 7 ⟩ +≡

```
func floorRat(v *big.Rat) int {
    q := new(big.Int).Quo(v.Num(), v.Denom())
    if new(big.Rat).SetInt(q).Cmp(v) > 0 {
        q.Sub(q, big.NewInt(1))
    }
    return int(q.Int64())
}
```

63. ⟨ Functions 7 ⟩ +≡

```
func ceilRat(v *big.Rat) int {
    q := new(big.Int).Quo(v.Num(), v.Denom())
    if new(big.Rat).SetInt(q).Cmp(v) < 0 {
        q.Add(q, big.NewInt(1))
    }
    return int(q.Int64())
}
```

64. The printed diagrams. The answers print their patterns as pictures with the labels written in. Here they are transcribed, one string per row, so that the program works on exactly the diagrams the book draws rather than on something it chose itself.

```
⟨ Declarations 6 ⟩ +≡
    var diagram7 = []string{"2777", "2661", "2431", "5531"}
    var diagrams8 = [][]string{
        {"38881", "37721", "35421", "66421"},
        {"28881", "23661", "23541", "77741"},
        {"28888", "23661", "23541", "77741"},
        {"17773", "12663", "12544", "88844"},
    }
```

65. Reading a diagram gives back both the pattern and its labeling, and checks on the way that the letters really do form a dissection.

```
⟨ Functions 7 ⟩ +≡
    func fromGrid(g []string)(pattern, []int) {
        p := pattern{m: len(g), n: len(g[0])}
        seen, order := map[byte]int{}, []byte{}
        for i := 0; i < p.m; i++ {
            for j := 0; j < p.n; j++ {
                ⟨ Start a new piece at (i, j) unless one is there already 66 ⟩
            }
        }
        ⟨ Check that the pieces tile the grid 67 ⟩
        perm := make([]int, len(p.rs))
        for _, c := range order {
            perm[int(c - '1')] = seen[c]
        }
        return p, perm
    }
```

66. ⟨ Start a new piece at (i, j) unless one is there already 66 ⟩ ≡

```
    c := g[i][j]
    if _, ok := seen[c]; ok {
        continue
    }
    r := rect{i, i + 1, j, j + 1}
    for r.r1 < p.m ∧ g[r.r1][j] ≡ c {
        r.r1++
    }
    for r.c1 < p.n ∧ g[i][r.c1] ≡ c {
        r.c1++
    }
    seen[c] = len(p.rs)
    p.rs = append(p.rs, r)
    order = append(order, c)
```

This code is used in section 65.

67. ⟨ Check that the pieces tile the grid 67 ⟩ ≡

```
for i := 0; i < p.m; i++ {
    for j := 0; j < p.n; j++ {
        r := p.rs[seen[g[i][j]]]
        if i < r.r0 ∨ i ≥ r.r1 ∨ j < r.c0 ∨ j ≥ r.c1 {
            log.Fatalf("%v is not a dissection at (%d,%d)", g, i, j)
        }
    }
}
```

This code is used in section 65.

68. Order seven. Answer 374(d) proves that an incomparable dissection of order at most seven reduces to the first of the two 4×4 motley patterns, with its regions labeled in one particular way. The census above says how much of that is forced by the pattern alone, and the linear programs settle the rest: of the nine labelings that survive the containment test, exactly one is feasible in any case at all.

```
⟨ Work out order seven 68 ⟩ ≡
    p, perm := fromGrid(diagram7)
    fmt.Printf("order␣7,␣the␣diagram␣of␣answer␣374(d):␣%s,␣labels␣%v\n", p, perm)
    ⟨ Check that no other labeling of the pattern is feasible 69 ⟩
    ⟨ Solve the 128 cases of order seven 70 ⟩
    ⟨ Solve the 128 cases again, for squares 74 ⟩
    ⟨ Count the optimal dissections directly 75 ⟩
```

This code is used in section 4.

69.

```
⟨ Check that no other labeling of the pattern is feasible 69 ⟩ ≡
    good := 0
    for _, q := range p.labelings() {
        for sgn := 0; sgn < 128; sgn++ {
            if ok, _ := p.chain(q, sgn, false).lpOpt(); ok {
                good++
                break
            }
        }
    }
    fmt.Printf("␣␣␣␣%d␣labelings␣pass␣the␣containment␣test,␣%d␣are␣feasible\n",
        len(p.labelings()), good)
```

This code is used in section 68.

70. Every case that has a solution at all, and what its cheapest solution costs.

```
⟨ Solve the 128 cases of order seven 70 ⟩ ≡
    feasible, results := 0, []int{}
    best, bestCase, bestV := 999, −1, []int(Λ)
    for sgn := 0; sgn < 128; sgn++ {
        pr := p.chain(perm, sgn, false)
        if ok, _ := pr.lpOpt(); ¬ok {
            continue
        }
        feasible++
        _, n, v := pr.ipOpt(300)
        results = append(results, n)
        if n < best {
            best, bestCase, bestV = n, sgn, v
        }
        if sgn ≡ 0 {
            ⟨ Report the case where all the heights come first 71 ⟩
        }
    }
    sort.Ints(results)
    ⟨ Report the extremes of the 128 cases 72 ⟩
```

This code is used in section 68.

71. Answer 375(a) works this case out by hand, and prints both the value and the dimensions that achieve it.

⟨ Report the case where all the heights come first 71 ⟩ ≡

```
w, h := p.wh(perm, v)
fmt.Printf("␣␣␣␣all␣the␣h's␣first:␣semiperimeter␣%d,␣with␣w␣=␣%v,␣h␣=␣%v\n",
    n, w, h)
```

This code is used in section 70.

72. ⟨ Report the extremes of the 128 cases 72 ⟩ ≡

```
w, h := p.wh(perm, bestV)
fmt.Printf("␣␣␣␣%d␣of␣the␣128␣cases␣are␣feasible\n", feasible)
fmt.Printf("␣␣␣␣smallest␣semiperimeter␣%d,␣in␣the␣case\n␣␣␣␣␣␣␣%s\n",
    best, caseName(7, bestCase))
fmt.Printf("␣␣␣␣␣␣␣w␣=␣%v,␣h␣=␣%v\n", w, h)
fmt.Printf("␣␣␣␣the␣two␣smallest␣values␣are␣%d␣and␣%d;␣then␣%d;␣the␣largest" +
    "␣is␣%d\n", results[0], results[1], results[2], results[len(results) − 1])
⟨ Check the alternating case 73 ⟩
```

This code is used in section 70.

73. The answer singles this one out as infeasible, and explains why: any case with $h_6 < w_3 < h_5$ needs $h_4 + h_5 < w_3 + w_4$, hence $h_4 < w_4$.

⟨ Check the alternating case 73 ⟩ ≡

```
alt := 0
for k := 0; k < 7; k += 2 {
    alt |= 1 ≪ uint(k)
}
ok, _ := p.chain(perm, alt, false).lpOpt( )
fmt.Printf("␣␣␣␣the␣alternating␣case␣%s\n␣␣␣␣␣␣␣is␣feasible:␣%v\n",
    caseName(7, alt), ok)
```

This code is used in section 72.

74. Adding one equation to each of the 128 problems asks for a square instead.

⟨ Solve the 128 cases again, for squares 74 ⟩ ≡

```
nsq, side, sideCase, sideV := 0, 999, −1, []int(Λ)
for sgn := 0; sgn < 128; sgn++ {
    pr := p.chain(perm, sgn, true)
    if ok, _ := pr.lpOpt(); ¬ok {
        continue
    }
    nsq++
    if _, n, v := pr.ipOpt(300); n/2 < side {
        side, sideCase, sideV = n/2, sgn, v
    }
}
w2, h2 := p.wh(perm, sideV)
fmt.Printf("␣␣␣␣%d␣of␣the␣128␣are␣feasible␣as␣squares;␣smallest␣side␣%d\n",
    nsq, side)
fmt.Printf("␣␣␣␣␣␣␣case␣%s\n␣␣␣␣␣␣␣w␣=␣%v,␣h␣=␣%v\n",
    caseName(7, sideCase), w2, h2)
```

This code is used in section 68.

75. Both optima are said to be unique, and that is a claim about the dissections themselves rather than about the cases, so it is checked by brute force: run through every way of splitting the semiperimeter among the four column widths and the four row heights.

⟨ Count the optimal dissections directly 75 ⟩ ≡

```
n1, sols := p.countAt(best, false)
n2, sqs := p.countAt(2 ∗ side, true)
fmt.Printf("␣␣␣␣dissections␣of␣semiperimeter␣%d:␣%d␣%v\n", best, n1, sols)
fmt.Printf("␣␣␣␣squares␣of␣side␣%d:␣%d␣%v\n", side, n2, sqs)
```

This code is used in section 68.

76. Order eight. With eight subrectangles the reduction is 4×5, and answer 375(b) names four labeled diagrams: a full-height column beside the 4×4 pattern or beside its transpose, and the first two motley 4×5 patterns of exercise 365. Each is worked out below in the answer's own labeling.

```
⟨ Work out order eight 76 ⟩ ≡
    for d, g := range diagrams8 {
        p, perm := fromGrid(g)
        ⟨ Solve the 256 cases of this diagram 77 ⟩
    }
    ⟨ Look at the case whose optimum is not an integer 79 ⟩
    ⟨ Count the smallest squares of each diagram 81 ⟩
    ⟨ Look for patterns the four diagrams miss 82 ⟩
    ⟨ Try every labeling of every pattern 84 ⟩
```

This code is used in section 4.

77.

```
⟨ Solve the 256 cases of this diagram 77 ⟩ ≡
    feasible, semi, side := 0, 0, 0
    var semiV, sideV []int
    for sgn := 0; sgn < 256; sgn++ {
        for _, sq := range []bool { false, true }{
            pr := p.chain(perm, sgn, sq)
            ok, lo := pr.lpOpt()
            if ¬ok {
                continue
            }
            if ¬sq {
                feasible++
            }
            ⟨ Improve the record for this diagram, if this case can 78 ⟩
        }
    }
    w, h := p.wh(perm, semiV)
    u, g2 := p.wh(perm, sideV)
    fmt.Printf("diagram␣%d␣(%s,␣motley␣%v):␣%d␣of␣the␣256␣cases␣feasible\n",
        d + 1, p, p.strict(), feasible)
    fmt.Printf("␣␣␣␣smallest␣semiperimeter␣%d,␣with␣w␣=␣%v,␣h␣=␣%v\n", semi, w, h)
    fmt.Printf("␣␣␣␣smallest␣square␣side␣␣␣%d,␣with␣w␣=␣%v,␣h␣=␣%v\n", side, u, g2)
```

This code is used in section 76.

78. The relaxation is a lower bound, so a case whose bound is already too big needs no branch and bound at all. That is what keeps this quick.

⟨ Improve the record for this diagram, if this case can 78 ⟩ ≡

```
bound := ceilRat(lo)
if sq ∧ side > 0 ∧ bound ≥ 2 * side {
    continue
}
if ¬sq ∧ semi > 0 ∧ bound ≥ semi {
    continue
}
got, n, v := pr.ipOpt(300)
if ¬got {
    continue
}
if sq {
    if side ≡ 0 ∨ n/2 < side {
        side, sideV = n/2, v
    }
} else if semi ≡ 0 ∨ n < semi {
    semi, semiV = n, v
}
```

This code is used in section 77.

79. "These linear programs usually have integer solutions; but sometimes they don't," says the answer, and gives the case where the optimum is 97/2. To see the fractional optimum as a dissection, double every gap in the chain: that scales the whole problem by two, so its integer optimum is twice the rational one.

⟨ Look at the case whose optimum is not an integer 79 ⟩ ≡

```
p, perm := fromGrid(diagrams8[1])
sgn := 0
for _, k := range []int { 2, 4, 5, 6 }{
    sgn |= 1 ≪ uint(k)
}
pr := p.chain(perm, sgn, false)
_, lp := pr.lpOpt( )
_, n, v := pr.ipOpt(300)
w, h := p.wh(perm, v)
fmt.Printf("diagram 2, in the case\n  %s\n", caseName(8, sgn))
fmt.Printf("     the linear program gives %v; in integers it is %d," +
    " with w = %v, h = %v\n", lp, n, w, h)
⟨ Double the gaps to see the fractional optimum 80 ⟩
```

This code is used in section 76.

80. ⟨ Double the gaps to see the fractional optimum 80 ⟩ ≡

```
twice := p.chain(perm, sgn, false)
for i := range twice.gap {
    twice.gap[i] *= 2
}
_, n2, v2 := twice.ipOpt(400)
w2, h2 := p.wh(perm, v2)
fmt.Printf("␣␣␣␣doubling␣every␣gap␣gives␣%d,␣that␣is␣%d/2," +
    "␣with␣w␣=␣%v,␣h␣=␣%v,␣all␣halved\n", n2, n2, w2, h2)
```

This code is used in section 79.

81. The smallest square of all is the one in the first diagram, and the answer says there is only one way to get it.

⟨ Count the smallest squares of each diagram 81 ⟩ ≡

```
for d, g := range diagrams8 {
    p, _ := fromGrid(g)
    for _, side := range []int { 27, 35, 36 }{
        if n, sols := p.countAt(2 * side, true); n > 0 {
            fmt.Printf("diagram␣%d:␣%d␣squares␣of␣side␣%d␣%v\n",
                d + 1, n, side, sols)
            break
        }
    }
}
```

This code is used in section 76.

82. The census found six patterns of order eight that support an incomparable dissection, while the answer draws four diagrams. The two it does not draw are the remaining ways to put a full-height column beside the 4×4 pattern: there are four of those, not two, because the column can sit beside any of the four essentially different orientations. They are new patterns, but not new answers—each repeats one of the first two diagrams exactly.

⟨ Look for patterns the four diagrams miss 82 ⟩ ≡

```
drawn := map[string]bool{}
for _, g := range diagrams8 {
    q, _ := fromGrid(g)
    drawn[q.canon()] = true
}
for _, p := range patterns(8, false) {
    if drawn[p.canon()] {
        continue
    }
    ⟨ Report p if it supports an incomparable dissection 83 ⟩
}
```

This code is used in section 76.

83. ⟨ Report p if it supports an incomparable dissection 83 ⟩ ≡

```
semi, side := 0, 0
for _, perm := range p.labelings() {
    if ok, _ := p.looseLP(perm, false); ¬ok {
        continue
    }
    for sgn := 0; sgn < 256; sgn++ {
        for _, sq := range []bool { false, true }{
            pr := p.chain(perm, sgn, sq)
            ok, lo := pr.lpOpt()
            if ¬ok {
                continue
            }
            ⟨ Improve the record for this pattern, if this case can 85 ⟩
        }
    }
}
if semi > 0 {
    fmt.Printf("undrawn pattern %s: semiperimeter %d, square side %d\n",
        p, semi, side)
}
```

This code is used in section 82.

84. The fourth diagram gives 47 where the answer says 56, and the diagrams were transcribed by hand; so the first thing to suspect is my copy of it. I added this check in November 2026, after a bug in another of these readings (exercise 334) had hidden behind checks that all shared one step. It shares nothing with the transcription: it takes every pattern of order eight and every labeling of it, and finds the best semiperimeter and square of each. If some labeling of some pattern had its best semiperimeter at 56, a slip in copying would be the likely story. None does: each of the six patterns that work at all has exactly one labeling that works, and the six give 44 or 47, never 56.

```
⟨ Try every labeling of every pattern 84 ⟩ ≡
    for _, p := range patterns(8, false) {
        works := 0
        for _, perm := range p.labelings() {
            if ok, _ := p.looseLP(perm, false); ¬ok {
                continue
            }
            semi, side := 0, 0
            for sgn := 0; sgn < 256; sgn++ {
                for _, sq := range []bool { false, true }{
                    pr := p.chain(perm, sgn, sq)
                    ok, lo := pr.lpOpt()
                    if ¬ok {
                        continue
                    }
                    ⟨ Improve the record for this pattern, if this case can 85 ⟩
                }
            }
            if semi > 0 {
                works++
                fmt.Printf("pattern␣%s,␣labeling␣%v:␣semiperimeter␣%d,␣square␣side␣%d\n",
                    p, perm, semi, side)
            }
        }
        if works > 1 {
            fmt.Printf("pattern␣%s␣has␣%d␣labelings␣that␣work\n", p, works)
        }
    }
```

This code is used in section 76.

85. ⟨ Improve the record for this pattern, if this case can 85 ⟩ ≡

```
bound := ceilRat(lo)
if (sq ∧ side > 0 ∧ bound ≥ 2 ∗ side) ∨ (¬sq ∧ semi > 0 ∧ bound ≥ semi) {
    continue
}
got, n, _ := pr.ipOpt(300)
if ¬got {
    continue
}
if sq {
    if side ≡ 0 ∨ n/2 < side {
        side = n/2
    }
} else if semi ≡ 0 ∨ n < semi {
    semi = n
}
```

This code is used in sections 83, 84.

86. Nine subrectangles. Answer 375(b) remarks that no smaller square than 27 can be incomparably dissected in integers, "because nine subrectangles would be too many." That is a statement about order nine, and the machinery above can decide it: run the census, keep the labelings that survive the loose test, and take the smallest that any of their 2^9 cases can reach. It is the longest computation here, and the answer is 28—one more than 27, which is exactly the remark.

⟨ Ask whether nine subrectangles could do better 86 ⟩ ≡

```
ps := patterns(9, false)
labelings, kept, best := 0, 0, 999
for _, p := range ps {
    for _, perm := range p.labelings() {
        labelings++
        ok, lo := p.looseLP(perm, true)
        if ¬ok ∨ ceilRat(lo) ≥ 2 * best {
            continue
        }
        kept++
        ⟨ Try the 512 cases of this labeling 87 ⟩
    }
}
fmt.Printf("order␣9:␣%d␣patterns,␣%d␣labelings,␣%d␣worth␣the␣linear" +
    "␣programs\n", len(ps), labelings, kept)
fmt.Printf("␣␣␣␣no␣square␣of␣side␣less␣than␣%d,␣so␣nine␣pieces␣cannot" +
    "␣beat␣27\n", best)
```

This code is used in section 4.

87. ⟨ Try the 512 cases of this labeling 87 ⟩ ≡

```
for sgn := 0; sgn < 512; sgn++ {
    ok, lo := p.chain(perm, sgn, true).lpOpt()
    if ¬ok {
        continue
    }
    if n := ceilRat(lo); n < 2 * best {
        best = (n + 1)/2
    }
}
```

This code is used in section 86.

88. A direct search. None of the theory above is needed to find an incomparable dissection of a given pattern: one can simply try every way of splitting a given semiperimeter among the columns and the rows, and test the pieces against each other. That is far too slow to prove anything about large cases, but it is a completely independent check of the small ones, and it is what counts the optima.

```
⟨ Functions 7 ⟩ +≡
    func (p pattern) countAt(total int, square bool)(int, [][]int) {
        n := 0
        var out [][]int
        for a := p.n; a ≤ total − p.m; a++ {
            if square ∧ 2 ∗ a ≠ total {
                continue
            }
            ⟨ Try every pair of compositions with these two sums 89 ⟩
        }
        return n, out
    }
```

89. Widths that repeat can be discarded at once, and so can heights, since two pieces of equal width are certainly comparable.

```
⟨ Try every pair of compositions with these two sums 89 ⟩ ≡
    var good [][]int
    for _, x := range compositions(a, p.n) {
        if distinct(p.widths(x)) {
            good = append(good, x)
        }
    }
    if len(good) ≡ 0 {
        continue
    }
    for _, y := range compositions(total − a, p.m) {
        h := p.heights(y)
        if ¬distinct(h) {
            continue
        }
        for _, x := range good {
            if incomparable(p.widths(x), h) {
                n++
                if len(out) < 6 {
                    out = append(out, append(append([]int(Λ), x...), y...))
                }
            }
        }
    }
```

This code is used in section 88.

90. ⟨ Functions 7 ⟩ +≡

```
func compositions(sum, parts int) [][]int {
    var out [][]int
    cur := make([]int, parts)
    var rec func(i, left int)
    rec = func(i, left int) {
        if i ≡ parts − 1 {
            if left ≥ 1 {
                cur[i] = left
                out = append(out, append([]int(Λ), cur...))
            }
            return
        }
        for v := 1; v ≤ left − (parts − 1 − i); v++ {
            cur[i] = v
            rec(i + 1, left − v)
        }
    }
    rec(0, sum)
    return out
}
```

91. ⟨ Functions 7 ⟩ +≡

```
func (p pattern) widths(x []int) []int {
    w := make([]int, len(p.rs))
    for i, r := range p.rs {
        for c := r.c0; c < r.c1; c++ {
            w[i] += x[c]
        }
    }
    return w
}
```

92. ⟨ Functions 7 ⟩ +≡

```
func (p pattern) heights(y []int) []int {
    h := make([]int, len(p.rs))
    for i, r := range p.rs {
        for c := r.r0; c < r.r1; c++ {
            h[i] += y[c]
        }
    }
    return h
}
```

93. ⟨ Functions 7 ⟩ +≡

```
func distinct(v []int) bool {
    for i := range v {
        for j := i + 1; j < len(v); j++ {
            if v[i] == v[j] {
                return false
            }
        }
    }
    return true
}
```

94. The definition itself: piece i fits inside piece j if its smaller dimension is no larger and its larger dimension is no larger either.

⟨ Functions 7 ⟩ +≡

```
func incomparable(w, h []int) bool {
    for i := range w {
        ai, bi := minmax(w[i], h[i])
        for j := i + 1; j < len(w); j++ {
            aj, bj := minmax(w[j], h[j])
            if (ai ≤ aj ∧ bi ≤ bj) ∨ (aj ≤ ai ∧ bj ≤ bi) {
                return false
            }
        }
    }
    return true
}
```

95. ⟨ Functions 7 ⟩ +≡

```
func minmax(a, b int)(int, int) {
    if a > b {
        return b, a
    }
    return a, b
}
```

96. The two patterns of order seven that the theory rejects should have no incomparable dissection of any size, and the search confirms that none turns up below a generous bound. It also finds the two optima of order seven from scratch, without any labeling or any linear program.

⟨ Search the patterns directly, with no theory at all 96 ⟩ ≡

```
for _, p := range patterns(7, false) {
    n, sols := 0, [][]int(Λ)
    total := 0
    for total = p.m + p.n; total ≤ 40 ∧ n ≡ 0; total++ {
        n, sols = p.countAt(total, false)
    }
    if n ≡ 0 {
        fmt.Printf("␣␣␣␣␣%s:␣nothing␣with␣semiperimeter␣40␣or␣less\n", p)
        continue
    }
    fmt.Printf("␣␣␣␣␣%s:␣%d␣with␣semiperimeter␣%d␣%v\n", p, n, total − 1, sols)
}
⟨ Check the smallest square of order eight directly 97 ⟩
```

This code is used in section 4.

97. ⟨ Check the smallest square of order eight directly 97 ⟩ ≡

```
q, _ := fromGrid(diagrams8[0])
for side := 4; side ≤ 27; side++ {
    if n, sols := q.countAt(2 * side, true); n > 0 {
        fmt.Printf("␣␣␣␣␣diagram␣1␣of␣order␣eight:␣%d␣squares,␣the␣smallest" +
            "␣of␣side␣%d␣%v\n", n, side, sols)
        break
    }
}
```

This code is used in section 96.

98. Index.

a: 37, 42, 43, 47, 55, 61, 88, 89, 95.
acc: 56.
Add: 56, 63.
ai: 94.
aj: 94.
all: 23, 24.
allowed: 52.
alt: 73.
append: 8, 9, 12, 16, 17, 18, 22, 29, 34, 37, 38, 39, 40, 61, 66, 70, 89, 90.
art: 48, 54, 55, 56, 57, 58.
atLeast: 40, 42.
atoms: 32, 33, 37, 39, 40, 42, 43.
a0: 31.
a1: 31.
b: 12, 19, 25, 26, 35, 36, 47, 55, 61, 95.
basis: 48, 50, 53, 54, 55, 56, 57, 58, 59.
best: 15, 53, 60, 61, 70, 72, 75, 86, 87.
bestCase: 70, 72.
bestV: 70, 72.
bestX: 60, 61.
bi: 94.
big: 39, 44, 48, 49, 50, 51, 53, 54, 56, 58, 59, 62, 63.
bit: 35.
bj: 94.
bound: 78, 85.
br: 61.
Builder: 19, 25, 35.
b0: 31.
b1: 31.
c: 20, 22, 34, 37, 39, 41, 44, 45, 46, 54, 58, 60, 65, 66, 91, 92.
canon: 15, 22, 23, 82.
cap: 45, 60.
caseName: 35, 72, 73, 74, 79.
cases: 28.
ceilRat: 60, 61, 63, 78, 85, 86, 87.
cell: 7, 8, 9, 10, 11.
cells: 4, 24.
chain: 28, 33, 69, 70, 73, 74, 77, 79, 80, 83, 84, 87.
Cmp: 53, 62, 63.
col: 50, 51, 52, 53.
cols: 32, 33, 39, 43.
compositions: 89, 90.
countAt: 75, 81, 88, 96, 97.
cur: 7, 8, 9, 15, 90.
c0: 6, 8, 9, 10, 11, 12, 13, 14, 16, 17, 18, 20, 26, 30, 34, 41, 46, 67, 91.
c1: 6, 9, 10, 11, 12, 13, 14, 16, 17, 18, 20, 26, 30, 34, 41, 46, 66, 67, 91.
d: 76, 77, 81.
Dance: 24.
Denom: 62, 63.
diagrams8: 64, 76, 79, 81, 82, 97.
diagram7: 64, 68.
distinct: 89, 93.
distinctIntervals: 13, 22, 23.
drawn: 82.
eq: 43, 47, 55.
f: 37, 51, 58.
Fatalf: 4, 67.
feasible: 70, 72, 77.
flag: 5.
flip: 15, 17.
floorRat: 61, 62.
fmt: 18, 23, 24, 25, 26, 27, 28, 35, 68, 69, 71, 72, 73, 74, 75, 77, 79, 80, 81, 83, 84, 86, 96, 97.
form: 32, 34, 37, 39.
Fprintf: 25, 26, 35.
frac: 61.
free: 9, 10.
fromGrid: 65, 68, 76, 79, 81, 82, 97.
g: 19, 20, 34, 65, 66, 67, 76, 81, 82.
gap: 32, 34, 37, 42, 80.
ge: 37, 42, 47, 55, 61.
good: 69, 89.
got: 24, 78, 85.
gridProblem: 24, 25.
g2: 77.
h: 39, 40, 46, 71, 72, 77, 79, 89, 92, 94.
heights: 89, 92.
hi: 33, 34, 35.
h2: 74, 80.
i: 7, 8, 10, 11, 18, 19, 20, 25, 26, 29, 30, 37, 39, 42, 43, 46, 49, 50, 51, 53, 54, 55, 56, 57, 58, 59, 65, 66, 67, 80, 90, 91, 92, 93, 94.
incomparable: 89, 94.
inside: 30, 31.
Int: 62, 63.
Ints: 70.
Int64: 62, 63.
ipOpt: 45, 70, 74, 78, 79, 80, 85.
IsInt: 61.

j: 7, 8, 10, 11, 18, 20, 25, 26, 30, 50, 51, 52, 54, 55, 56, 57, 58, 59, 61, 65, 66, 67, 93, 94.
k: 15, 19, 29, 30, 32, 33, 34, 35, 36, 37, 39, 40, 58, 73, 79.
kept: 86.
Knuth, Donald Ervin: 1.
labelings: 28, 29, 69, 83, 84, 86.
le: 47, 55, 61.
left: 90.
len: 7, 9, 22, 23, 27, 28, 29, 33, 39, 46, 54, 60, 65, 66, 69, 72, 86, 89, 91, 92, 93, 94.
limit: 7.
live: 27, 28.
lo: 60, 77, 78, 83, 84, 85, 86, 87.
log: 4, 67.
looseLP: 28, 39, 83, 84, 86.
lp: 79.
lpOpt: 28, 44, 69, 70, 73, 74, 77, 79, 83, 84, 87.
lrow: 37, 39, 42, 43, 47, 54, 60, 61.
m: 4, 6, 7, 8, 9, 12, 14, 16, 17, 18, 19, 21, 23, 24, 25, 27, 33, 39, 41, 48, 50, 52, 53, 54, 55, 56, 57, 58, 59, 65, 66, 67, 88, 89, 96.
main: 4.
make: 7, 12, 19, 29, 33, 37, 39, 42, 43, 46, 54, 59, 61, 65, 90, 91, 92.
mask: 34, 39, 40, 41.
minmax: 94, 95.
minus: 42.
mn: 23.
mode: 5.
modes: 4, 5.
mot: 23.
Mul: 51, 58.
N: 48, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59.
n: 6, 7, 8, 9, 12, 14, 16, 17, 18, 19, 21, 23, 24, 25, 26, 27, 28, 33, 34, 39, 41, 46, 48, 54, 55, 58, 59, 60, 61, 65, 66, 67, 70, 71, 74, 78, 79, 81, 85, 87, 88, 89, 96, 97.
needStrict: 21, 22.
Neg: 56, 59.
new: 49, 50, 51, 53, 54, 56, 58, 59, 62, 63.
NewInt: 62, 63.
NewReader: 24.
NewXCC: 24.
normal: 15, 18.
nsq: 74.
Num: 62, 63.
n1: 75.
n2: 75, 80.
ok: 28, 30, 39, 44, 60, 66, 69, 70, 73, 74, 77, 83, 84, 86, 87.
order: 65, 66.
out: 7, 8, 21, 22, 29, 88, 89, 90.
p: 12, 13, 14, 15, 16, 17, 18, 19, 21, 22, 23, 27, 28, 29, 30, 33, 34, 39, 40, 41, 46, 50, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 79, 80, 81, 82, 83, 84, 86, 87, 88, 89, 91, 92, 96.
paint: 9, 11.
paintByte: 19, 20.
Parse: 5.
parts: 90.
pattern: 6, 7, 8, 12, 13, 14, 15, 16, 17, 18, 19, 21, 29, 33, 39, 41, 46, 65, 88, 91, 92.
patterns: 21, 27, 82, 84, 86, 96.
pc: 46.
perm: 28, 29, 30, 33, 34, 39, 40, 46, 65, 68, 70, 71, 72, 73, 74, 76, 77, 79, 80, 83, 84, 86, 87.
piece: 41.
pivot: 50, 53, 57.
plus: 42.
pr: 33, 34, 37, 38, 39, 40, 43, 44, 45, 70, 74, 77, 78, 79, 83, 84, 85.
Printf: 23, 24, 27, 28, 68, 69, 71, 72, 73, 74, 75, 77, 79, 80, 81, 83, 84, 86, 96, 97.
Println: 23.
problem: 32, 33, 37, 39, 43, 44, 45.
ps: 27, 86.
q: 15, 16, 17, 53, 62, 63, 69, 82, 97.
Quo: 50, 53, 62, 63.
r: 12, 13, 14, 16, 17, 18, 19, 20, 34, 41, 46, 50, 51, 54, 55, 66, 67, 91, 92.
rat: 49, 55, 58.
Rat: 39, 44, 48, 49, 50, 51, 53, 54, 56, 58, 59, 62, 63.
rec: 7, 9, 29, 30, 60, 61, 90.
rect: 6, 7, 8, 9, 16, 17, 18, 20, 66.
red: 23.
reduced: 12, 22, 23.
results: 70, 72.
role: 33, 34.
rot: 15, 16.
row: 19, 53, 61.
rows: 37, 41, 44, 45, 54, 60.
rs: 6, 12, 13, 14, 16, 17, 18, 19, 22, 29, 30, 33, 34, 37, 38, 39, 40, 41, 44, 45, 46, 60, 61, 65, 66, 67, 91, 92.
run: 52, 56, 58.

r0: 6, 8, 9, 10, 11, 12, 13, 14, 16, 17, 18, 20, 25, 26, 30, 34, 41, 46, 67, 92.
r1: 6, 9, 10, 11, 12, 13, 14, 16, 17, 18, 20, 25, 26, 30, 34, 41, 46, 66, 67, 92.
s: 15, 18.
sc: 13.
seen: 21, 22, 23, 65, 66, 67.
semi: 77, 78, 83, 84, 85.
semiV: 77, 78.
sense: 47, 55, 61.
Set: 50, 51, 59.
SetInt: 62, 63.
SetInt64: 49.
sgn: 28, 33, 35, 69, 70, 74, 77, 79, 80, 83, 84, 87.
side: 74, 75, 77, 78, 81, 83, 84, 85, 97.
sideCase: 74.
sideV: 74, 77, 78.
Sign: 50, 51, 52, 53, 56, 57, 58.
sign: 55.
sizes: 27.
Slice: 18.
sols: 75, 81, 96, 97.
Solutions: 24.
solveIP: 45, 60.
solveLP: 39, 44, 54, 60.
sort: 18, 70.
Sprintf: 18, 27.
sq: 77, 78, 83, 84, 85.
sqs: 75.
square: 32, 33, 38, 39, 88.
squareRow: 38, 39, 43.
sr: 13.
strict: 14, 22, 23, 27, 28, 77.
String: 5, 19, 25, 35.
strings: 19, 24, 25, 35.
Sub: 51, 58, 62.
sum: 90.
T: 50, 51, 52, 53, 54, 55, 56, 57, 58, 59.
t: 21, 22, 27, 28, 29, 30, 33, 34, 35, 36, 39, 48, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59.
tab: 48, 50, 52, 54.
tilings: 7, 21, 23.
total: 88, 89, 96.
twice: 80.
u: 77.
used: 29, 30.
usedC: 12.
usedR: 12.
v: 11, 39, 44, 46, 60, 61, 62, 63, 70, 71, 74, 78, 79, 90, 93.
v2: 80.
w: 33, 39, 40, 46, 71, 72, 77, 79, 91, 94.
wh: 46, 71, 72, 74, 77, 79, 80.
wi: 33, 34, 35.
widths: 89, 91.
works: 84.
Write: 19.
WriteByte: 19, 25, 26.
WriteString: 36.
w2: 74, 80.
x: 24, 59, 60, 61, 89, 91.
y: 89, 92.

⟨ Add the row that makes it a square 38 ⟩ Used in section 37
⟨ Append the kth dimension to the chain 34 ⟩ Used in section 33
⟨ Ask `ssxcc` to enumerate the same tilings 24 ⟩ Used in section 23
⟨ Ask whether nine subrectangles could do better 86 ⟩ Used in section 4
⟨ Check that no other labeling of the pattern is feasible 69 ⟩ Used in section 68
⟨ Check that the pieces tile the grid 67 ⟩ Used in section 65
⟨ Check the alternating case 73 ⟩ Used in section 72
⟨ Check the smallest square of order eight directly 97 ⟩ Used in section 96
⟨ Clear column *col* in row i 51 ⟩ Used in section 50
⟨ Count the motley dissections, exercise 365 23 ⟩ Used in section 4
⟨ Count the optimal dissections directly 75 ⟩ Used in section 68
⟨ Count the smallest squares of each diagram 81 ⟩ Used in section 76
⟨ Declarations 6 ⟩ Used in section 4
⟨ Double the gaps to see the fractional optimum 80 ⟩ Used in section 79
⟨ Drive any artificial variable out of the basis 57 ⟩ Used in section 56
⟨ Enter row i of the tableau 55 ⟩ Used in section 54
⟨ Find the first empty cell, or record a tiling 8 ⟩ Used in section 7
⟨ Functions 7 ⟩ Used in section 4
⟨ Give label $k+1$ to piece i, if that is allowed 30 ⟩ Used in section 29
⟨ Improve the record for this diagram, if this case can 78 ⟩ Used in section 77
⟨ Improve the record for this pattern, if this case can 85 ⟩ Used in sections 83, 84
⟨ Keep p if it is a pattern we want 22 ⟩ Used in section 21
⟨ Look at the case whose optimum is not an integer 79 ⟩ Used in section 76
⟨ Look for patterns the four diagrams miss 82 ⟩ Used in section 76
⟨ Phase I: drive the artificial variables to zero 56 ⟩ Used in section 54
⟨ Phase II: minimise the objective 58 ⟩ Used in section 54
⟨ Pivot on the best row of column *col*, or give up 53 ⟩ Used in section 52
⟨ Read off the solution 59 ⟩ Used in section 54
⟨ Read the command line 5 ⟩ Used in section 4
⟨ Report the case where all the heights come first 71 ⟩ Used in section 70
⟨ Report the extremes of the 128 cases 72 ⟩ Used in section 70
⟨ Report p if it supports an incomparable dissection 83 ⟩ Used in section 82
⟨ Say that the widths rise and the heights fall 40 ⟩ Used in section 39
⟨ Say whether p can be made incomparable 28 ⟩ Used in section 27
⟨ Search the patterns directly, with no theory at all 96 ⟩ Used in section 4
⟨ Solve the 128 cases again, for squares 74 ⟩ Used in section 68
⟨ Solve the 128 cases of order seven 70 ⟩ Used in section 68
⟨ Solve the 256 cases of this diagram 77 ⟩ Used in section 76
⟨ Start a new piece at (i,j) unless one is there already 66 ⟩ Used in section 65
⟨ Take a census of the patterns of orders 7 and 8 27 ⟩ Used in section 4
⟨ Take the solution, or branch on a fractional variable 61 ⟩ Used in section 60
⟨ Try every labeling of every pattern 84 ⟩ Used in section 76
⟨ Try every pair of compositions with these two sums 89 ⟩ Used in section 88
⟨ Try every rectangle with that cell at its corner 9 ⟩ Used in section 7
⟨ Try the 512 cases of this labeling 87 ⟩ Used in section 86

⟨ Work out order eight 76 ⟩ Used in section 4
⟨ Work out order seven 68 ⟩ Used in section 4
⟨ Write the options with rows *r0* to *r1* 26 ⟩ Used in section 25
⟨ Write the separator before the kth dimension 36 ⟩ Used in section 35

Incomparable dissections

	Section	Page
Introduction	1	1
Dissections and their reductions	6	4
The census	23	10
Labelings	29	13
Cases	32	15
Exact linear programming	47	21
The printed diagrams	64	28
Order seven	68	30
Order eight	76	33
Nine subrectangles	86	39
A direct search	88	40
Index	98	44